



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 1

Software-Engineering mit KI-Agenten **Grundlagen**

Disclaimer

In den Folien wird „die KI“ an einigen Stellen vermenschlicht (Anthropomorphismus). Damit lassen sich bestimmte Sachverhalte besser erklären, z.B. „KI als Kollegin“.

Achtung: Ich gehe von folgenden Annahmen über Sprachmodelle und KI-Agenten aus (Stand September 2026):

1. KI hat **kein Verantwortungsbewusstsein**, die Konsequenzen ihres Handelns spielen für sie keine Rolle. Die Verantwortung liegt bei den Menschen, welche die KI bedienen!
2. KI ist eine laufende Software, **sie simuliert** Einfühlungsvermögen, Persönlichkeit und andere menschliche Eigenschaften, KI hat kein Bewusstsein



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 1.1

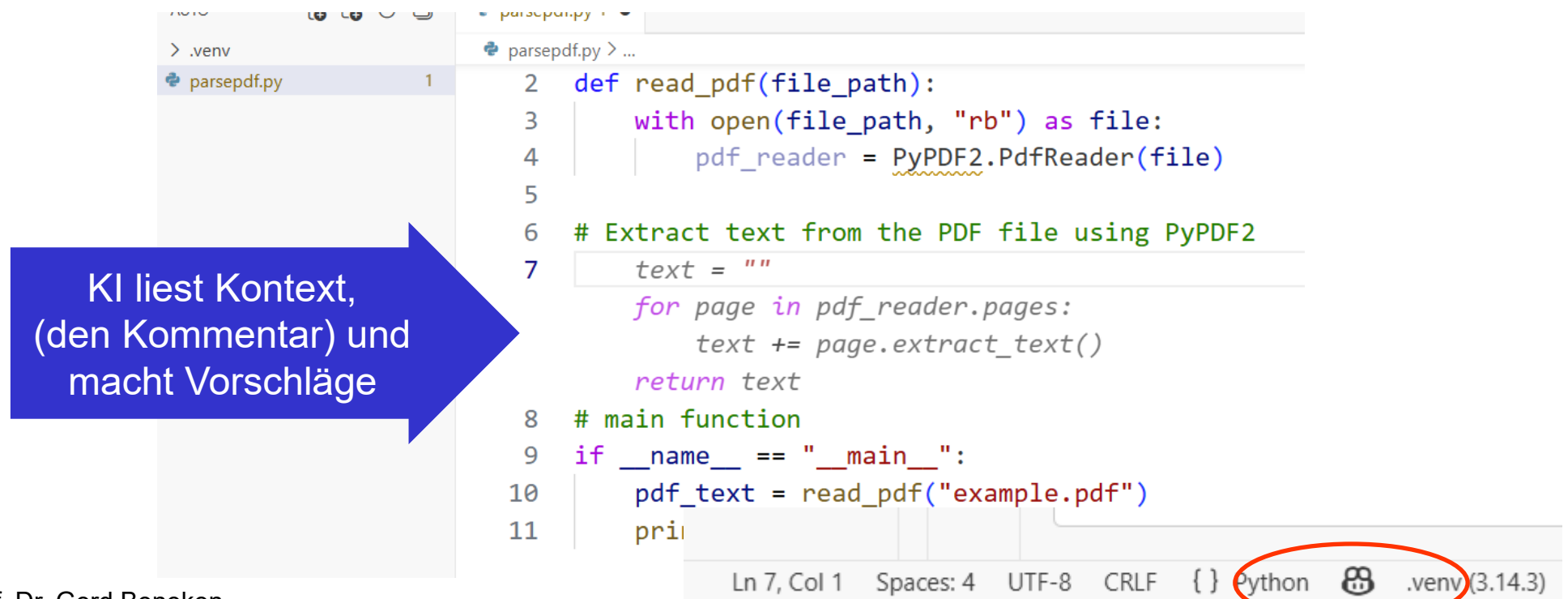
Zusammenarbeit mit einem KI-Agenten

Zusammenarbeit mit einer KI im Software-Engineering

1. Intelligente Autovervollständigung
2. KI als Werkzeug: Chat Based Programming, Code Review, ...
3. KI als Sparringspartner: Interaktive KI
4. KI als Kollegin: KI-Agenten wie Claude Code und Codex

2021: GitHub Copilot – Autovervollständigung

- KI macht beim Tippen mehrere Vorschläge für Code (TAB-Taste)
- KI kann aus Kommentar Code-Blöcke generieren (TAB-Taste)
- Kontext = offene Datei, ggf. mehr, Prompts = nur indirekt, Entwicklerin verwendet die TAB-Taste



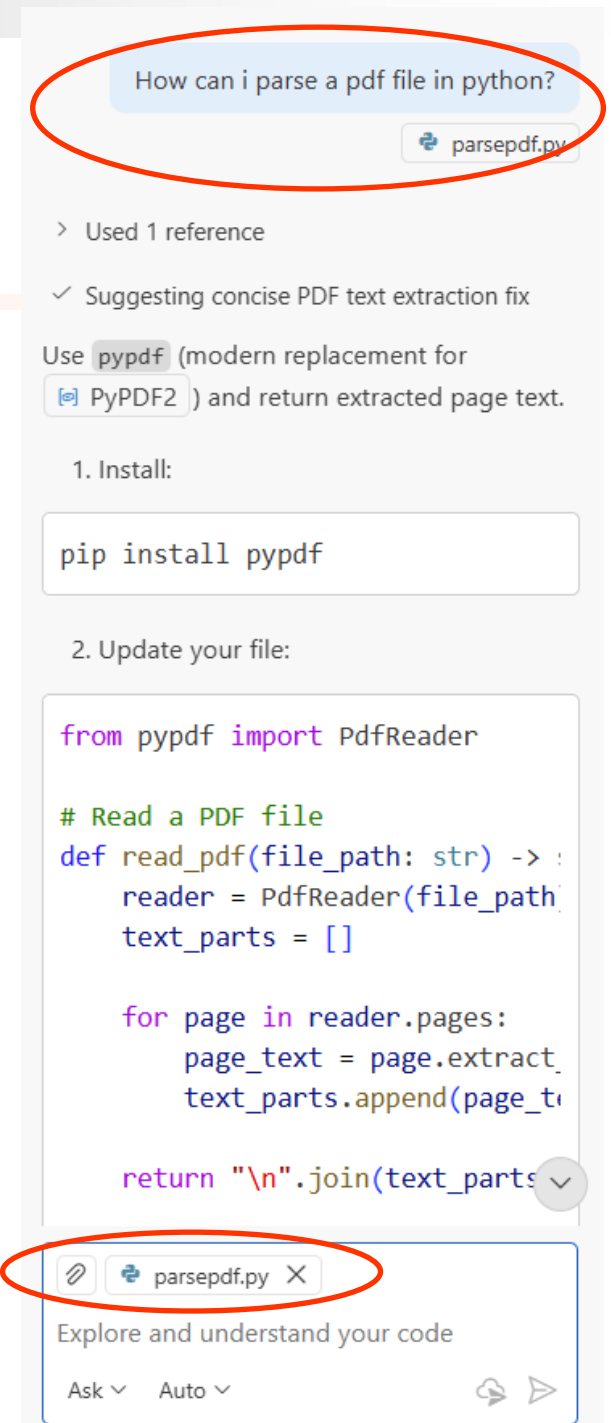
2023: KI Als Werkzeug (noch GitHub Copilot)

Frage

Antwort

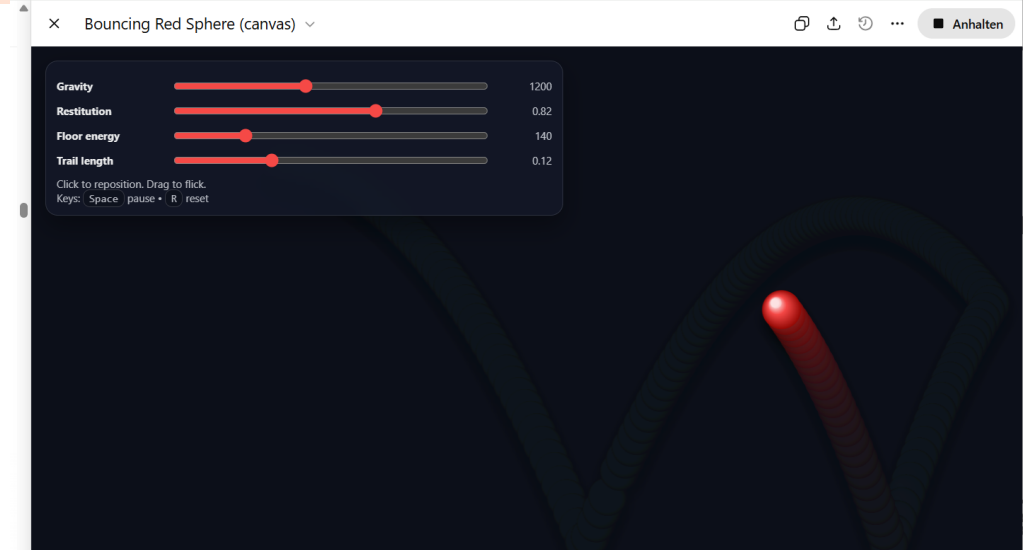
- **Chat-Fenster in IDE:** Weiterentwicklung manuell und über Prompts der Entwicklerin
- Entwicklerin steuert
 - stellt Fragen zum Code (erkläre mir ...)
 - kopiert Fehlermeldung in Chat
 - bittet um Refactoring-Vorschläge
 - Codegenerierung in größerem Umfang möglich
- Kontext = Steuerbar (ggf. auch ganze Codebase), Entwicklerin muss KI-Generat ggf. noch manuell kopieren, Kontext groß genug für typische Codebase

Kontext



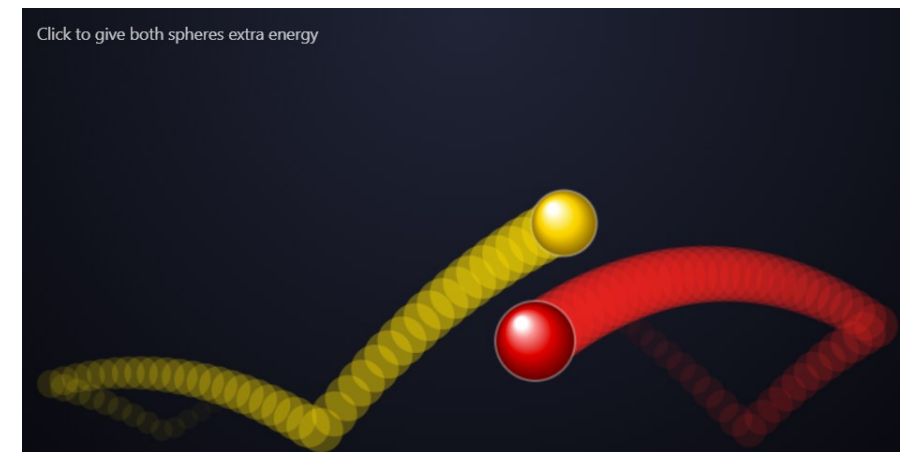
Beispiel für „Chat Coding“ (Aus Kim, Yegge: „Vibe Coding“) einfach in ChatGPT kopieren ...

- Selber testen mit ChatGPT 5:
`please write a javascript app in
an artifacts window that animates
a bouncing red sphere. Leave a
trail behind it Add gravity and
energy when it hits the floor.”`



- Ausprobieren und dann
Änderungswünsche mitteilen ...

- add a second yellow ball
- Make the red ball a little larger
- Fix Bug “....”



2024: KI als Sparringspartner (Diskussion mit KI)

- **Interaktive Diskussion mit KI-Agent** (stellt Fragen, gibt Feedback)
- Bedürfnisse, Anforderungen, Architektur werden mit KI **diskutiert, verfeinert, präzisiert, korrigiert**
 - Review, Plausibilisierung, Kritik im Gespräch (KI gibt Feedback)
 - Diskussion und Plausibilisierung von Anforderungen
Beispiel: „grill-me-skill“ von Mark Pocock, KI findet selbstständig Lücken / Annahmen / Unklarheiten
 - KI dokumentiert das Diskussionsergebnis als „Plan“ oder in der Spezifikation oder ...
 - Wichtig: Der KI das „Warum“ erklären
- Kontext: Agent hat (lesenden) Zugriff auf Spezifikation, Architektur, Codebase als Markdown im Repository

← ☒ Akteure ☒ Format ☒ Umfang ✓ Submit →

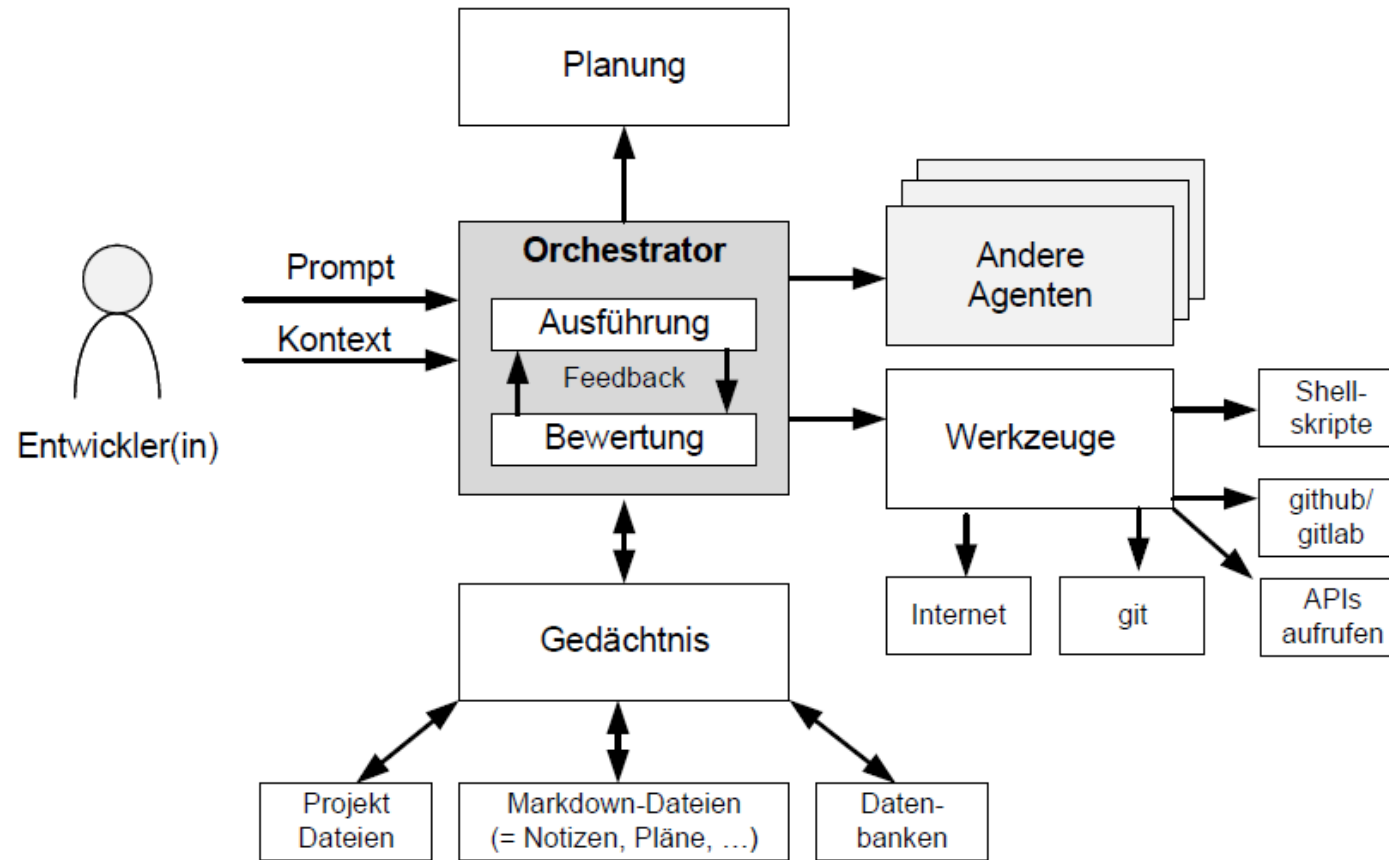
Welche Domänenbereiche sollen abgedeckt werden?

- › 1. ☐ Customer-Verwaltung
Anlegen, Abfragen, Suchen von Kunden.
 - 2. ☐ Reservierungs-Verwaltung
Erstellen, Abfragen, Filtern, Stornieren von
 - 3. ☐ Ladestations-Verwaltung
Stationen auflisten, einzeln abfragen, Kosten
 - 4. ☐ System & Health
Service-Verfügbarkeit prüfen inkl. DB- und Ra
 - 5. ☐ Type something
- Submit

2025: KI als Kollegin z.B. Codex, Claude Code

- Agent = Software, das **autonom** komplexere Aufgaben erledigen kann
 - **Planungskomponente**: Entwirft selbst expliziten Plan und setzt den dann um
 - **Ausführungskomponente**: führt den Plan aus, erzeugt Code, Spezifikationen, UML, Web-Oberflächen etc., auch Code-Generierung
 - **Prüfkomponente**: z.B. QS von Code gibt hinweise für die Generierungskomponente, Aufgabe bereits erledigt (Abbruchkriterium erfüllt)?
 - **Testumgebungen**: z.B. github codespaces, dev-container, ... = Entwicklungs- und Testumgebung steuerbar, z.B. zum Unit-Test ausführen
 - **Tools**: Kann Dateien selber ändern, in git Branches, oder in github pull-requests auslösen, download z.B. von API-Spezifikation (Open API)
- Beispiele: Claude Code und Codex – Kommandozeilen Tools (CLI)
- Kontext: Agent hat lesenden / schreiben Zugriff auf Projektdaten, Repository, ggf. Issue-Tracker, über Tools Zugriff auf beliebige Informationsquellen z.B. Internet-Recherche

Was ist ein KI-Agent?





Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 1.2

Allgemeine Regeln

Zusammenarbeit mit KI-Agenten

- Allgemeine KI-Leitplanken definieren
 - Was genau darf die KI (-> Welche Tools sind erlaubt)?
 - Worauf darf die KI lesend / schreibend zugreifen (-> Projektverzeichnis)
 - Bestimmte Dateien darf die KI nicht sehen, z.B. „.env“, Compliance beachten
- Immer in einem (Git-) Repository arbeiten
 - Zwischenstände über Commits sichern, häufiges push
 - Git = Undo Funktion, wenn ein Experiment nicht funktioniert hat
 - Git = Dokumentation von Änderungen (der KI)
 - Welche Dateien wurden geändert?
 - Was genau wurde geändert (Diff)?
- Auch Prompts unter Versionsverwaltung stellen (-> Reproduzierbarkeit)

Zusammenarbeit mit KI-Agenten

- Professionelle Entwicklungsumgebung aufsetzen
 - Build- und Dependency-Management (-> SBOM)
 - Lokale Testumgebung (z.B. mit Docker / Testcontainers)
 - CI / CD inklusive Testautomatisierung, Statische Code Analyse, ...
 - Gitlab / Github mit Issues, Feature-Branches, Merge-Requests
- Code-Architektur festlegen: Modularität
 - Subsysteme / Komponenten / Services definieren und deren Schnittstelle festlegen
 - Begründung:
 - KI sieht und ändert NUR Innensicht EINER Komponente,
 - Kontext deutlich kleiner (Tokens sparen)
 - Effekt kontrollierbarer (da auf eine Komponente begrenzt)
- Leitplanken für das Projekt definieren
 - Erlaubte Frameworks und Bibliotheken
 - Regeln für „guten“ / „sauberen“ Code, zusätzlich zu SonarQube, Linter, ...
 - Entwurfsprinzipien, z.B. Minimalitätsprinzip, Fail Fast, Least Priviledge, ...

Markdown als zentrales Dokumentformat

- Start einer Markdown-Datei mit Meta-Informationen (yml)

```
---
```

```
Titel: „Super Tolle Spezifikation“
```

```
Autor: Gerd Beneken
```

```
Stand: 13.09.2026
```

```
Status: in Arbeit
```

```
---
```

- Gliederung über # Hauptüberschrift, ## Unterüberschrift, ### ...
- Tabellen, Aufzählungen
- Verweise auf andere Dateien [Beispiele] (.\beispiele\README.md)
- UML-Modelle, Mindmaps als Text in PlantUML oder Mermaid (PlantUML Plugins in Visual Studio Code, IntelliJ, gitlab, ...)



Agiles Software-Entwicklungsprojekt

Gerd Beneken

Kapitel 1.3

Beispiel Claude Code

Claude Code

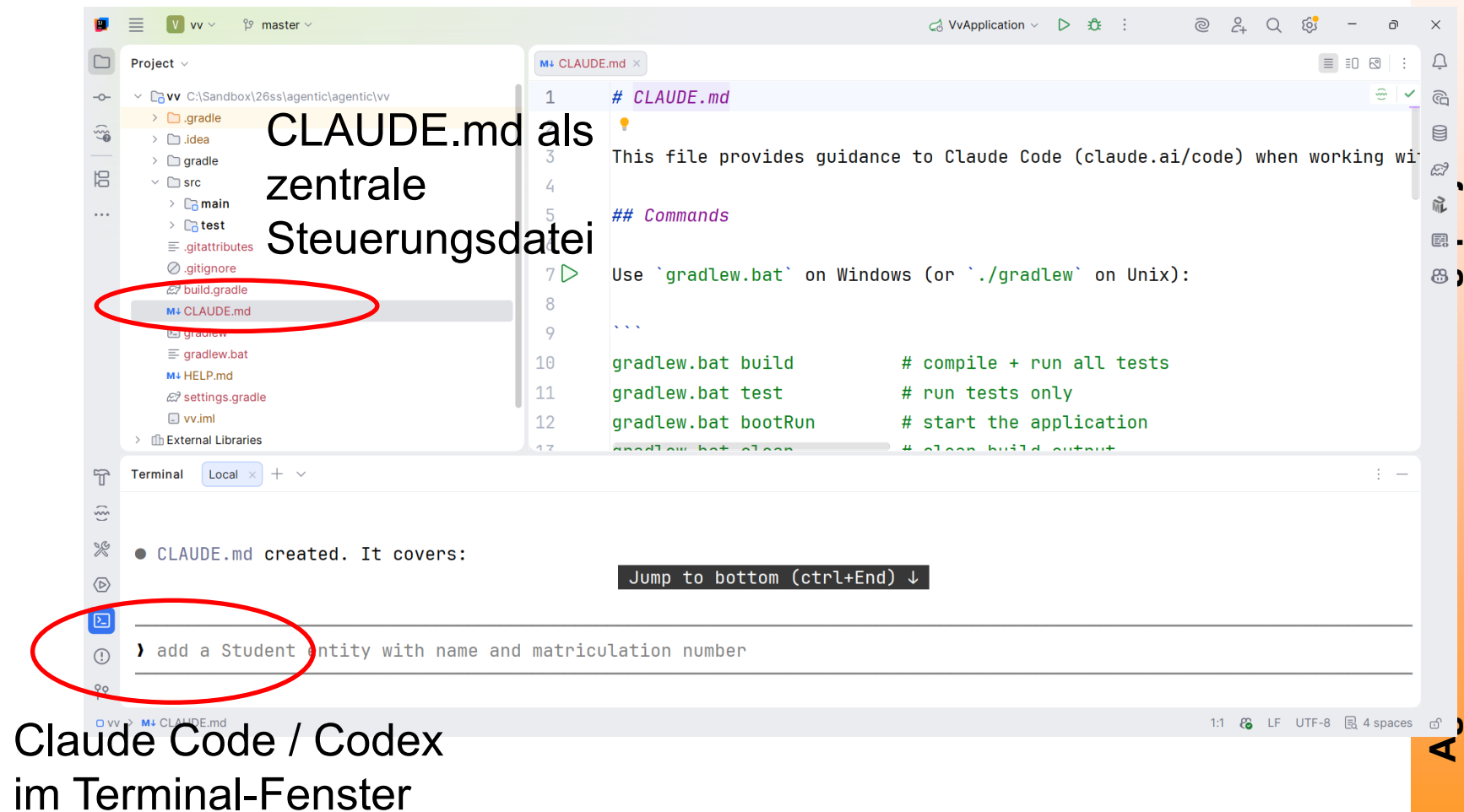
[Wir arbeiten mit der 20\$-Subscription von Anthropic]

KI-gestütztes CLI, das direkt im Projektverzeichnis Code liest, schreibt, testet und committet — ohne Copy-Paste zwischen Chat und IDE.

- Versteht gesamte Codebasis als Kontext (nicht nur einzelne Dateien)
- Führt Build-Tools, Tests und Git-Operationen selbstständig aus
- Konfigurierbar über **CLAUDE.md** — projektspezifische Architektur- und Coding-Konventionen
- Erweiterbar durch Skills (wiederverwendbare Anweisungen) und Subagents (parallele Teilaufgaben)
- Installation auf Kommandozeile
- Ähnliche Alternative: Codex von OpenAI, OpenCode, Aider, ...
- Weitere Alternativen: Antigravity von Google, Kiro von Amazon, Cursor,

Start mit Claude Code (oder Codex)

- Git Repository bereitstellen (!)
`git init`
- Projekt in IDE anlegen
- Claude Code im Terminal ausführen, dann `/init`
- Claude Code legt dann **CLAUDE.md** Datei an
 - = Architektur des Projekts (Frameworks)



Zentrale Festlegungen in CLAUDE.md

Architecture

Spring Boot 4.1.0 application, Java 21, package root `de.thro.vv`.

Layer stack:

- **Web:** Spring MVC (`spring-boot-starter-webmvc`) — REST controllers
- **Persistence:** Spring Data JPA + Hibernate — entities and repositories
- **DB migrations:** Flyway — SQL scripts live in `src/main/resources/db/migration/`
- **Messaging:** Spring AMQP — RabbitMQ producers and consumers
- **Validation:** Bean Validation on request DTOs

Databases:

- Development/test: H2 in-memory (H2 console enabled)
- Production: Microsoft SQL Server (`mssql-jdbc` + `flyway-sqlserver`)

Claude Code erzeugt einen Plan.

=> Wir sind „Projektleiterin“, überwachen, steuern, entscheiden

Plan: JPA Entities & Repositories for Reservation Service

Context

The project is a fresh Spring Boot 4.1.0 scaffold (Java 21, `de.thro.vv`). The data model in `datamodel.md` is the persistence layer for a charging-station reservation system. No entities, repositories, or SQL migrations exist yet. Goal: create all JPA entities, Spring Data repositories, a Flyway V1 migration, and minimal `application.properties` config so the application starts and tests pass.

Files to Create

1. `de.thro.vv.entity.ReservationStatus` (enum)

`src/main/java/de/thro/vv/entity/ReservationStatus.java`

Values: `REQUESTED, CONFIRMED, ACTIVE, COMPLETED, CANCELLED, EXPIRED`

2. `de.thro.vv.entity.Customer`

`src/main/java/de/thro/vv/entity/Customer.java`

Field	Type	Notes
`customerId`	`Long`	`@Id @GeneratedValue(IDENTITY)`
`firstName`	`String`	`@NotBlank`

Ergebnis

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the package structure `de.thro.vv` with sub-packages `entity` and `repository`. The `entity` package contains `ChargingStation`, `Customer`, `Reservation`, `ReservationEventLog`, and `ReservationStatus`. The `repository` package contains `ChargingStationRepository`, `CustomerRepository`, `ReservationEventLogRepository`, and `ReservationRepository`.
- Code Editor:** Displays the `Reservation` entity class with the following code:


```

public class Reservation {
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long reservationId;

    @ManyToOne(fetch = FetchType.LAZY, optional = false) 3 usages
    @JoinColumn(name = "station_id")
    private ChargingStation station;

    @ManyToOne(fetch = FetchType.LAZY, optional = false) 3 usages
    @JoinColumn(name = "customer_id")
    private Customer customer;

    @NotNull 3 usages

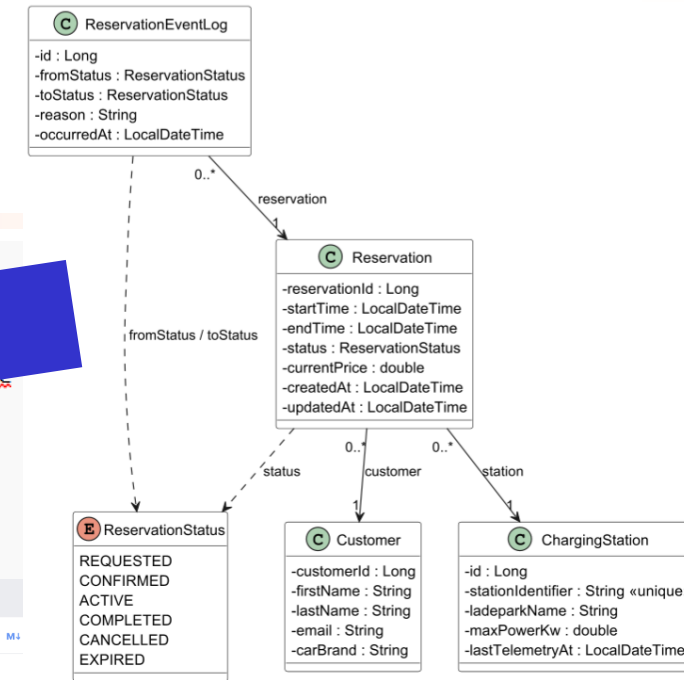
```
- Terminal:** Shows the output of the Flyway migration:


```

Infrastructure: V1__init.sql creates all 4 tables with FK constraints; application.properties wires H2 as the dev
datasource with Flyway + ddl-auto=validate.

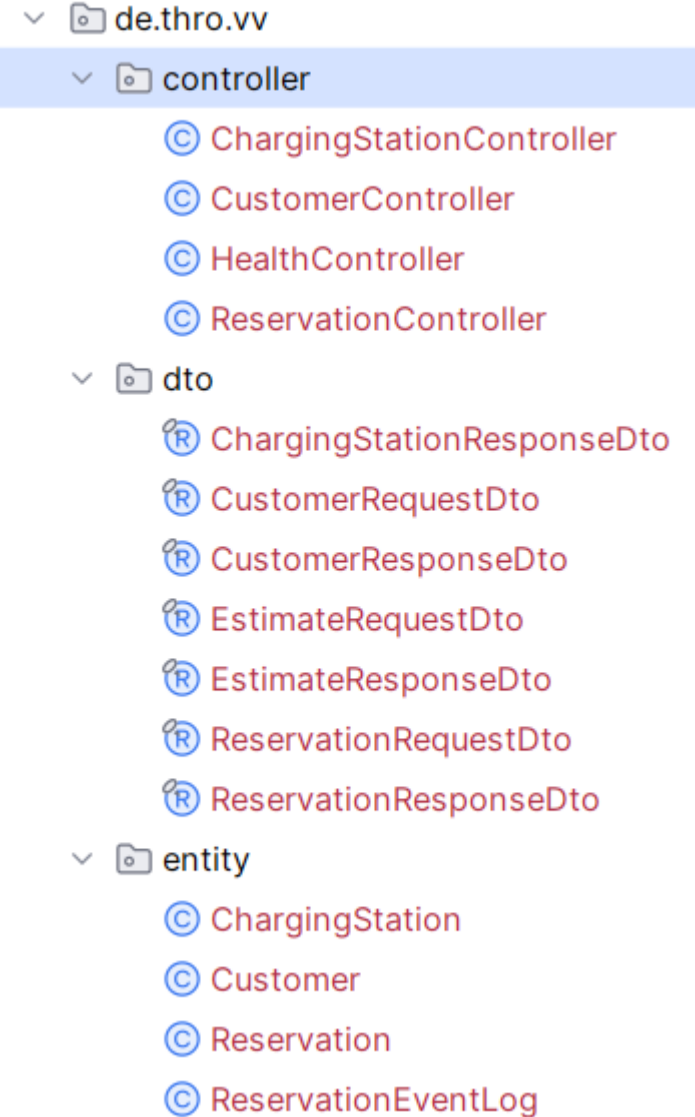
* Cooked for 4m 16s

```
- Commit Message:** The commit message is `jpa-entities-repositories-setup`.



Iteratives Vorgehen

- Alle Anwendungsfälle, Endpunkte auf einmal?
 - Möglich, aber das Ergebnis erschlägt
 - **Prozess iterativ**, erst Ergebnis ausprobieren, daraus neue Fragen und Lücken, dann Anforderungen hinzufügen, dann wieder generieren
 - Einen Anwendungsfall / Endpunkt erzeugen, dann Testfälle erzeugen, dann Code Review, Commit, Merge-Request etc.
- Schritte der KI einzeln kontrollieren, Grund:
 - Macht mehr Spaß, kleine Erfolgserlebnisse
 - Mehr gefühlte **Verantwortung** für das Ergebnis
 - So eher Sonder-, Rand- und Fehlerfälle finden, da **Lösung tiefer durchdacht** wird



Interaktive Verfeinerung mit KI-Agent

Ich würde gerne Anwendungsfälle für das vorliegende Projekt erstellen. Könnten wir diese interaktiv entwerfen. Die Anwendungsfälle will ich in der Datei @spec\use-cases.md speichern

- Claude stellt Fragen zu Akteuren, Daten und den Inhalten von Anwendungsfällen
- Präsentiert Optionen, Varianten
- Update aller (der meisten) Dateien mit den entsprechenden Informationen

← ☒ Vertragsfelder ☐ UC-Umfang ✓ Submit →

Was soll ein Vertrag beinhalten?

- › 1. [] Vertragsnummer + Laufzeit (Start/Ende)
Identifikation und Gültigkeitszeitraum des Vertrags.
 - 2. [] Tarifmodell / Preisstaffel
Z.B. Flatrate, kWh-Preis oder monatliche Grundgebühr.
 - 3. [] Zugelassene Ladestationen / Ladeparks
Welche Stationen der Kunde im Rahmen des Vertrags nutzen darf.
 - 4. [] Status (ACTIVE, SUSPENDED, TERMINATED)
Lebenszyklus des Vertrags.
 - 5. [] Type something
- Next

Externe Datenquellen verwenden

in der Datei `@spec\externalservice.md` findest Du eine OpenAPI Spezifikation. Bitte erzeuge einen Service, der diesen externen Dienst aufrufen kann.

- = Abruf der Spezifikation aus dem Netz
`https://vv.azurewebsites.net/swagger/v1/swagger.json`
- Dazu Code und / oder Doku erzeugen
- Datenquellen aus dem Netz
 - Anleitungen (z.B. welche Tabellen benötigt das Auth. Framework?)
 - Spezifikationen z.B. in OpenAPI

```
public interface HeadquarterService { 2 usages 1 implementation

    /**
     * Liefert den aktuellen Preis pro kWh für eine Ladestation.
     *
     * @param stationId Identifizier der Ladestation (station_identifizier)
     * @return aktueller Preis in EUR/kWh
     */
    double getCurrentPricePerKwh(String stationId); 1 usage 1 implementation

    /**
     * Schätzt die Gesamtkosten für einen Ladevorgang.
     *
     * @param stationId Identifizier der Ladestation
     * @param expectedKwh erwartete Energiemenge in kWh (0.1 - 200)
     * @return geschätzte Gesamtkosten in EUR
     */
    double estimateChargingCost(String stationId, double expectedKwh);
```

Tools

Kategorie	Beispiele	Permission-relevant
Dateisystem lesen	Read, LS, Glob, Grep	Nein — immer erlaubt
Dateisystem schreiben	Write, Edit, MultiEdit	Ja — wird geprüft
Shell-Ausführung	Bash	Ja — höchstes Risiko
Aufgabenverwaltung	TodoRead, TodoWrite	Nein
Web	WebSearch, WebFetch	Nein

- KI-Agent kann Effekt in realer Welt erzeugen
 - Aufruf von Skripten auf der lokalen Maschine (z.B. ./gradlew test)
 - Steuerung von Docker, Docker Compose
 - Zugriff auf Dienste im Internet, z.B. GitHub, GitLab, Slack
- Werkzeugeinsatz kann genehmigt oder verweigert werden
- Werkzeuge über Protokolle wie MCP aufrufbar
 - LLM findet aus Prompt selbstständig Service und Parameter
 - Viele Werkzeuge => Kontext aufgebraucht

```
claude mcp add mein-tool --command "node /pfad/zum/server.js"
```

Beispiel mit glab [TODO]

```
claude mcp add ...  
gitlab/gitlab:latest glab mcp serve
```

- glab = Gitlab CLI
- Zugriff auf Issues, Wiki, Repository, Meilensteine, etc.
 - Neue Issues / Feature Branches / Merge-Requests erzeugen
 - Vorhandene Issues konsumieren
 - Informationen aus Wiki auslesen
- KI kann selbstständig Merge-Request bearbeiten, Code Review zwingend von Projektmitgliedern (!)
- KI kann Issues aus Anforderungen / Architektur erzeugen, Plan => Umsetzungsplan

Skills: Funktionsumfang erweitern Dazulernen

```
.claude/  
└─ skills/  
    └─ spring-boot-architecture/  
        └─ SKILL.md          ← Schichtenregeln, Namenskonventionen  
    └─ flyway-migrations/  
        └─ SKILL.md          ← Migrationsstrategie, Beispiele  
    └─ docker-compose/  
        └─ SKILL.md          ← Volume-Mapping, Healthchecks
```

- = wiederverwendbare Markdown-Dateien mit Anweisungen, Konventionen und Codebeispielen
 - Claude liest sie bei Bedarf und wendet sie auf die aktuelle Aufgabe an
- Erstelle einen neuen Controller. Beachte dabei den Skill "spring-boot-architecture".
- Quellen für Skills
 - Manuell ergänzen während der Entwicklung
 - Claude „dazulernen“ lassen (Claude erzeugt den Skill)
 - Bibliotheken aus dem Internet nutzen (mit Vorsicht)